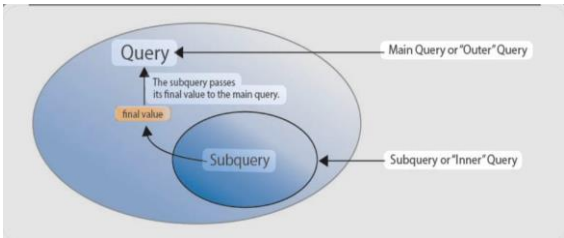
NYKAA BEAUTY STORE MANAGEMENT SYSTEM

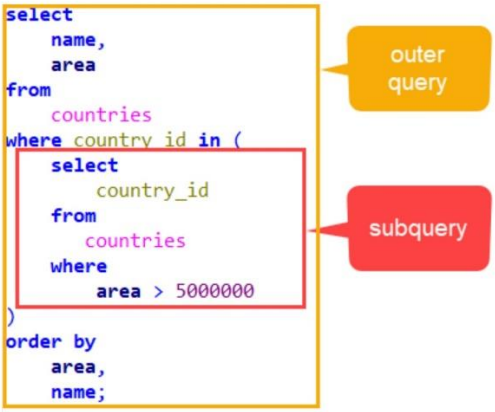
DBMS – ADVANCED SQL QUERY SYSTEM

1.subqueries and nested queries

Definition: A subquery is a query written inside another SQL query. It is used to retrieve data based on the result of another query.



basic structure



types of subqueries

- Single Row Subquery
- Multiple Row Subquery
- Multiple Column Subquery
- Correlated Subquery
- Nested Subquery

2.find product above average price

Purpose: Displays products whose price is greater than the average product price**.**

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price > (SELECT AVG(Price)
  4                 FROM Product);

PRODUCT_ID PRODUCT_NAME                                         PRICE
---------- -------------------------------------------------- ----------
       202 Serum                                                1200
       205 Perfume                                              1500
```

2.1product below average price

Purpose: Displays products whose price is lower than the average product price.

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price < (SELECT AVG(Price)
  4                 FROM Product);

PRODUCT_ID PRODUCT_NAME                                       PRICE
---------- -------------------------------------------------- ----------
       201 Face Wash                                             500
       203 Lipstick                                              800
       204 Shampoo                                               700
       206 Moisturizer                                           600
```

3.identify customer with maximum purchases

Purpose: Finds the highest total purchase amount among all customers**.**

step 1 calculate customer-wise purchase

```
SQL> SELECT C.Customer_ID,
  2         C.Customer_Name,
  3         SUM(O.Total_Amount) AS Total_Purchase
  4  FROM Customer C
  5  JOIN Orders O
  6  ON C.Customer_ID = O.Customer_ID
  7  GROUP BY C.Customer_ID, C.Customer_Name;

CUSTOMER_ID CUSTOMER_NAME                                 TOTAL_PURCHASE
----------- -------------------------------------------------- --------------
        101 Akshaya                                             3100
        102 Priya                                               2000
        103 Divya                                               4800
        104 Kavya                                               1900
        105 Sneha                                               1600
```

step 2 find the maximum purchase

```
SQL> SELECT MAX(Total_Purchase)
  2  FROM
  3  (
  4      SELECT Customer_ID,
  5             SUM(Total_Amount) AS Total_Purchase
  6      FROM Orders
  7      GROUP BY Customer_ID
  8  );

MAX(TOTAL_PURCHASE)
-------------------
               4800
```

3.1 display customer details with maximum purchase

Purpose: Displays complete customer details of the customer who has made the maximum total purchase.

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.Customer_Name,
  4      SUM(O.Total_Amount) AS Total_Purchase
  5  FROM Customer C
  6  JOIN Orders O
  7      ON C.Customer_ID = O.Customer_ID
  8  GROUP BY
  9      C.Customer_ID,
 10      C.Customer_Name
 11  HAVING SUM(O.Total_Amount) =
 12  (
 13      SELECT MAX(Total_Purchase)
 14      FROM
 15      (
 16          SELECT SUM(Total_Amount) AS Total_Purchase
 17          FROM Orders
 18          GROUP BY Customer_ID
 19      )
 20  );

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_PURCHASE
----------- -------------------------------------------------- --------------
        103 Divya                                                        4800
```

the innermost query

SELECT SUM(Total_Amount)

FROM Orders

GROUP BY Customer_ID;

the next query

SELECT MAX(Total_Purchase)

FROM

(

SELECT SUM(Total_Amount) AS Total_Purchase

FROM Orders

GROUP BY Customer_ID

);

4. complex business queries

4.1 customer who placed orders above average order value

Purpose: Displays customers who have placed at least one order above the average order value.

```
SQL> SELECT Customer_ID,
  2         Customer_Name
  3  FROM Customer
  4  WHERE Customer_ID IN
  5  (
  6      SELECT Customer_ID
  7      FROM Orders
  8      WHERE Total_Amount >
  9      (
 10          SELECT AVG(Total_Amount)
 11          FROM Orders
 12      )
 13  );

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
        102 Priya
        103 Divya
```

5. find products whose sales are above average

Purpose: Displays products whose total sales value is greater than the average sales value of products.

```
SQL> SELECT P.Product_ID,
  2         P.Product_Name,
  3         SUM(OD.Quantity * P.Price) AS Total_Sales
  4  FROM Product P
  5  JOIN Order_Details OD
  6  ON P.Product_ID = OD.Product_ID
  7  GROUP BY P.Product_ID, P.Product_Name
  8  HAVING SUM(OD.Quantity * P.Price) >
  9  (
 10      SELECT AVG(Product_Sales)
 11      FROM
 12      (
 13          SELECT SUM(OD.Quantity * P.Price) AS Product_Sales
 14          FROM Product P
 15          JOIN Order_Details OD
 16          ON P.Product_ID = OD.Product_ID
 17          GROUP BY P.Product_ID
 18      )
 19  );

PRODUCT_ID PRODUCT_NAME                                       TOTAL_SALES
---------- -------------------------------------------------- -----------
       202 Serum                                                     2400
       203 Lipstick                                                  2400
       205 Perfume                                                   4500
```

6. customer purchase report

Purpose: Generates a customer purchase report showing total orders, total purchase amount and average order value.

```
SQL> SELECT C.Customer_ID,
  2         C.Customer_Name,
  3         COUNT(O.Order_ID) AS Total_Orders,
  4         SUM(O.Total_Amount) AS Total_Purchase,
  5         AVG(O.Total_Amount) AS Average_Order
  6  FROM Customer C
  7  JOIN Orders O
  8  ON C.Customer_ID = O.Customer_ID
  9  GROUP BY C.Customer_ID, C.Customer_Name
 10  ORDER BY Total_Purchase DESC;

CUSTOMER_ID CUSTOMER_NAME                                       TOTAL_ORDERS
----------- -------------------------------------------------- ------------
TOTAL_PURCHASE AVERAGE_ORDER
-------------- -------------
        103 Divya                                                          2
          4800          2400

        101 Akshaya                                                        2
          3100          1550

        102 Priya                                                          1
          2000          2000


CUSTOMER_ID CUSTOMER_NAME                                       TOTAL_ORDERS
----------- -------------------------------------------------- ------------
TOTAL_PURCHASE AVERAGE_ORDER
-------------- -------------
        104 Kavya                                                          1
          1900          1900

        105 Sneha                                                          1
          1600          1600
```

7. advanced product sales report

Purpose: Displays product-wise sales details, including total orders, quantity sold, product price and total sales.

```
SQL> SELECT P.Product_ID,
  2         P.Product_Name,
  3         COUNT(DISTINCT OD.Order_ID) AS Total_Orders,
  4         SUM(OD.Quantity) AS Total_Quantity,
  5         P.Price,
  6         SUM(OD.Quantity * P.Price) AS Total_Sales
  7  FROM Product P
  8  JOIN Order_Details OD
  9  ON P.Product_ID = OD.Product_ID
 10  GROUP BY P.Product_ID, P.Product_Name, P.Price
 11  ORDER BY Total_Sales DESC;

PRODUCT_ID PRODUCT_NAME                                       TOTAL_ORDERS
---------- -------------------------------------------------- ------------
TOTAL_QUANTITY      PRICE TOTAL_SALES
-------------- ---------- -----------
       205 Perfume                                                       2
             3       1500        4500

       202 Serum                                                         2
             2       1200        2400

       203 Lipstick                                                      2
             3        800        2400


PRODUCT_ID PRODUCT_NAME                                       TOTAL_ORDERS
---------- -------------------------------------------------- ------------
TOTAL_QUANTITY      PRICE TOTAL_SALES
-------------- ---------- -----------
       201 Face Wash                                                     2
             3        500        1500

       204 Shampoo                                                       1
```

8. category-wise advanced sales report

Purpose: Generates a category-wise sales report showing total products, quantity sold and total sales for each beauty category.

```
SQL> SELECT C.Category_ID,
  2         C.Category_Name,
  3         COUNT(DISTINCT P.Product_ID) AS Total_Products,
  4         SUM(OD.Quantity) AS Total_Quantity,
  5         SUM(OD.Quantity * P.Price) AS Total_Sales
  6  FROM Category C
  7  JOIN Product P
  8  ON C.Category_ID = P.Category_ID
  9  JOIN Order_Details OD
 10  ON P.Product_ID = OD.Product_ID
 11  GROUP BY C.Category_ID, C.Category_Name
 12  ORDER BY Total_Sales DESC;

CATEGORY_ID CATEGORY_NAME                  TOTAL_PRODUCTS TOTAL_QUANTITY
----------- ------------------------------ -------------- --------------
TOTAL_SALES
-----------
          1 Skincare                                    3              7
       5100

          4 Fragrance                                   1              3
       4500

          2 Makeup                                      1              3
       2400


CATEGORY_ID CATEGORY_NAME                  TOTAL_PRODUCTS TOTAL_QUANTITY
----------- ------------------------------ -------------- --------------
TOTAL_SALES
-----------
          3 Haircare                                    1              2
       1400
```